

Name: Adasooriya Adasooriya

Student Reference Number: 10965435

Module Code: PUSL2023 - 25/SP/M	Module Name: PUSL2023 Mobile App Development (25/SP/M)
Coursework Title: Final Report Submission & Source Code	
Deadline Date: 9th April 2026	Member of staff responsible for coursework: Mr. Diluka Wijesinghe
Programme: BSc (Hons) Computer Science	
Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook .	
Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts. Adasooriya Adasooriya – 10965435 Bengamu Perera - 10965579 Ranaweera Ranaweera - 10965595 Rammandalage Rammandala - 10965589 Senarath Bandara - 10965463 Kapuge Dewmini- 10965483 Ramanayake Mihisara – 10965569 <i>We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.</i> Signed on behalf of the group:  Individual assignment: <i>I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.</i> Signed:  Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence. I *have used/not used translation software. If used, please state name of software.....	
Overall mark _____% Assessors Initials _____ Date _____	



GOODDEEDS

Community Volunteering
Mobile Application

Track your good deeds.
Inspire the world.



Group 40

Table of Contents

	Page No:
Chapter 01:	
1.1 Introduction -----	05
1.2 Existing Systems and Problem Definition -----	06
1.3 Project Aims and Objectives -----	08
1.4 Scope of the Project -----	10
Chapter 02:	
2.1 Requirement Gathering Techniques -----	11
2.2 Functional Requirements -----	12
2.3 Non-Functional Requirements -----	15
2.4 Features of the Application	
Chapter 03:	
3.1 Use Case Diagram -----	17
3.2 High-Level Diagram -----	19
3.3 EER Diagram -----	21
3.4 User Interface of the Developed System -----	23
Chapter 04:	
4.1 Development Methodology -----	30
4.2 Technologies and Tools Used -----	31
4.3 Future Implementation -----	32
Chapter 05:	
5.1 Individual contribution -----	33
5.2 Github commit history -----	35
5.3 Project Source Code Link -----	36
Reference List -----	37

List of Figures

Figure 1.2.1: Existing systems Challenges -----	07
Figure 1.3.2: Project objectives for GoodDeeds app -----	09
Figure 2.2.1: GoodDeeds application requirements -----	14
Figure 2.3.2: Feature of the GoodDeeds App -----	16
Figure 3.1.1: Use Case Diagram -----	18
Figure 3.3.3: High level Diagram -----	20
Figure 3.3.3: EER: Illustrates the EER Diagram of the GoodDeeds app -----	22
Figure 3.4.1: Welcome Screen -----	23
Figure 3.4.2: Registration Screen -----	23
Figure 3.4.3: Login Screen -----	23
Figure 3.4.4: Firebase Authentication -----	23
Figure 3.4.5: Forgot Password Screen. -----	24
Figure 3.4.6: Password Reset Email Received. -----	24
Figure 3.4.7: Firebase Firestore Database Structure. -----	25
Figure 3.4.8: Organizer dashboard Screen-----	25
Figure 3.4.9: Create Event Screen -----	25
Figure 3.4.10: Participants Screen -----	25
Figure 3.4.11: Organizer Profile Screen -----	26
Figure 3.4.12: Firestore Events Collection -----	27
Figure 3.4.13: Discover Events -----	27
Figure 3.4.14: Event Details Screen -----	27
Figure 3.4.15: My Events -----	27
Figure 3.4.16: User Profile-----	27
Figure 3.4.17: Edit Profile-----	27
Figure 3.4.18: Community Screen -----	28
Figure 3.4.19: Public Profile -----	28
Figure 3.4.20: My Friends Screen-----	28
Figure3.4.21: My Friends -----	28
Figure 3.4.22: Firebase firestore user_lookup collection. -----	29

Chapter 01

1.1 Introduction

This project focuses on the development of a mobile application named **GoodDeeds** – Community Volunteering Mobile Application, designed to support and manage community service activities in an organized and efficient manner. The application was developed using Flutter and Firebase technologies to ensure accessibility, scalability, and real-time data management.

The system provides a role-based platform for two main types of users: volunteers and organizers. Volunteers can register, log in securely, browse available events, and participate in community activities. They can also track their attendance, monitor completed and pending events, and view their contribution through impact points and activity history.

Organizers are provided with functionalities to create and manage events, update event details, upload event images, and monitor participant engagement. Additionally, a controlled attendance verification process is implemented to ensure accurate tracking of volunteer participation.

The application integrates Firebase Authentication for secure user login, Cloud Firestore for storing application data, and Firebase Storage for managing media content such as event images. Social interaction features, including comments, likes, and user connections, are also incorporated to enhance community engagement.

Overall, the **GoodDeeds** application provides a centralized, user-friendly, and secure platform that simplifies event discovery, participation, attendance tracking, and social interaction within community service activities.

1.2 Existing systems and problem definition

At present, community service and volunteering activities are primarily organized through informal and fragmented platforms such as social media networks, messaging applications, and manual record-keeping methods. While these platforms provide basic communication and coordination capabilities, they lack a structured and standardized approach for managing volunteering activities. As a result, information is often scattered across multiple channels, leading to inconsistencies, miscommunication, and inefficiencies in organizing and participating in events. This fragmented nature makes it challenging for both volunteers and organizers to maintain clarity, reliability, and proper documentation throughout the lifecycle of community service activities.

Existing systems do not provide a centralized platform where users can conveniently discover, join, and monitor volunteering opportunities. Volunteers frequently encounter difficulties in identifying suitable events that align with their interests, availability, and location. In many cases, event details are either incomplete or not clearly communicated, making it harder for individuals to make informed decisions. Additionally, confirming participation is often handled through informal methods, which can result in confusion or lack of proper acknowledgment. On the other hand, organizers face significant challenges in managing participant data, coordinating communication, verifying attendance, and maintaining accurate records of volunteer contributions. These limitations reduce the overall effectiveness and scalability of community service initiatives.

Furthermore, there is no efficient or standardized mechanism to track individual social impact or to encourage sustained engagement in volunteering activities. Most existing platforms do not offer features such as participation history, contribution tracking, performance summaries, or social interaction among volunteers. Without these functionalities, volunteers may find it difficult to assess their own contributions or feel recognized for their efforts. This lack of visibility and engagement can lead to decreased motivation and lower retention rates among participants. Additionally, the absence of transparency in tracking activities limits accountability and reduces trust in the system.

In addition, the lack of a reliable attendance verification mechanism further complicates the management of volunteering events. Current methods often rely on manual processes, such as paper-based records or unverified digital confirmations, which are prone to errors, manipulation, and data loss. These unstructured approaches are not only time-consuming but also inefficient, especially when dealing with large numbers of participants. Consequently, ensuring the accuracy and authenticity of participation records becomes a major challenge for organizers.

Therefore, there is a clear need for a centralized and integrated system that can effectively manage all aspects of community service activities in a structured, consistent, and user-friendly manner. The proposed **GoodDeeds** application aims to address these challenges by providing a unified platform that supports efficient event management, seamless participation tracking, reliable attendance verification, and enhanced social engagement. By integrating these functionalities into a single system, the application improves transparency, accountability, and overall user experience, ultimately fostering a more organized and impactful volunteering environment.



Figure 1.2.1: Existing systems Challenges

1.3 Project Aims and Objectives

- **Aim**

The primary aim of this project was to develop a reliable, efficient, and user-friendly system for managing and coordinating community service activities. The proposed system focuses on providing a centralized digital platform that simplifies the overall process of organizing, discovering, and participating in volunteering events. It is specifically designed to support both volunteers and organizers by enhancing communication, improving accessibility, and ensuring effective management of event-related activities. Additionally, the system aims to promote transparency, accountability, and continuous engagement in community service through structured and well-integrated features.

- **Objectives**

The objectives of the project were as follows:

- To develop a secure user authentication system that enables safe and reliable user registration and login.
- To implement role-based access and navigation for both volunteers and organizers to ensure appropriate system functionality.
- To design and develop a user-friendly interface that enhances usability, accessibility, and overall user experience.
- To allow organizers to efficiently create, update, and manage community service events.
- To enable users to easily browse, search, and join available volunteering opportunities based on their preferences.
- To implement an effective attendance verification system to ensure accurate and reliable participation tracking.
- To provide users with the ability to monitor their completed and pending volunteering activities.
- To develop a contribution tracking system that records impact points and maintains a detailed activity history.

- To integrate social interaction features such as comments, likes, and user connections to improve engagement.
- To ensure secure storage, processing, and management of data using cloud-based technologies
- To develop the application as a cross-platform system that is compatible with multiple devices and operating systems.

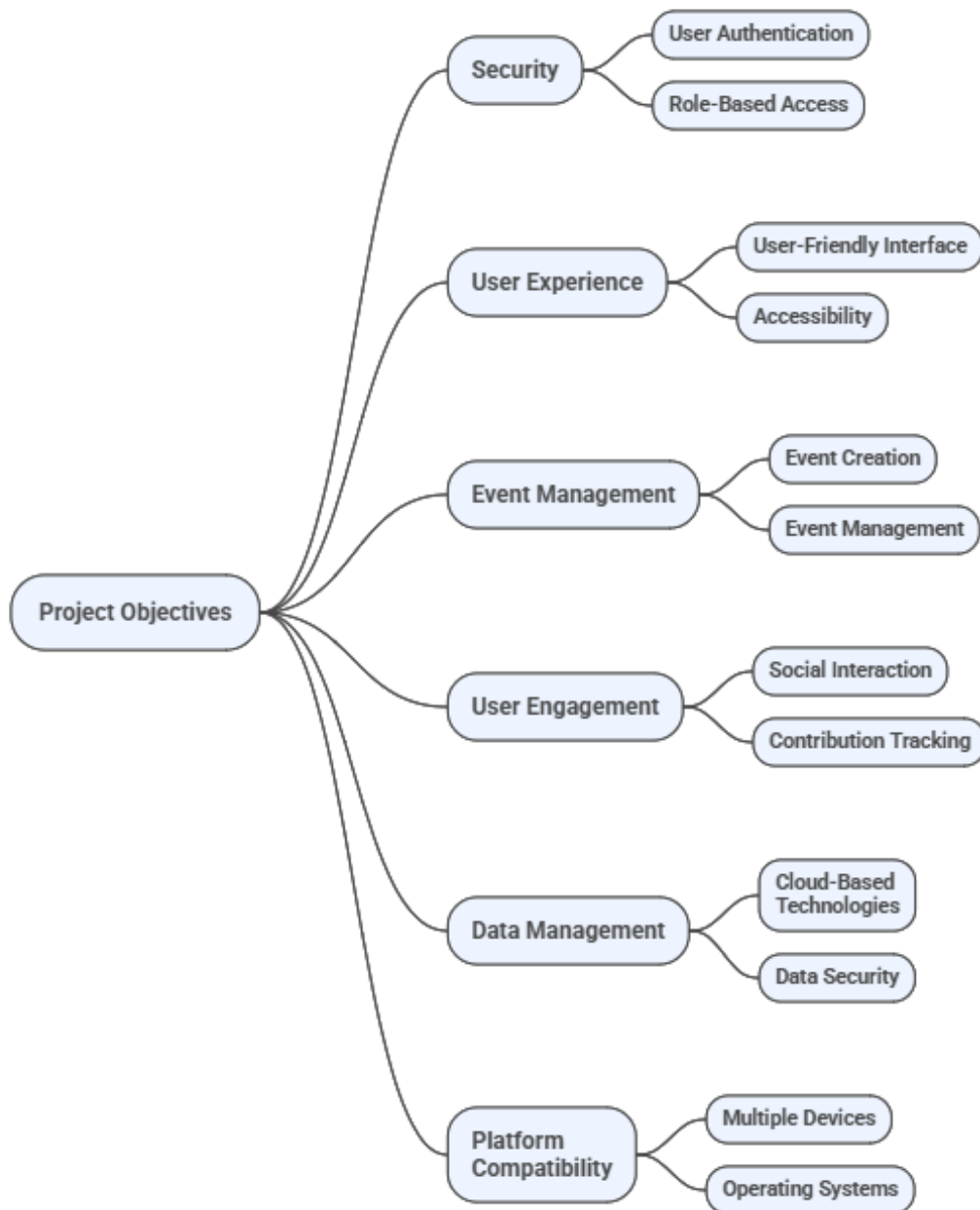


Figure 1.3.2: Project objectives for GoodDeeds app.

1.4 Scope of the Project

The scope of this project is focused on the design and development of a mobile application that supports the management of community service and volunteering activities. The system provides a centralized platform for both volunteers and organizers to interact, manage events, and track participation efficiently.

The implemented scope included secure account registration and login, role-based user access, event creation and editing by organizers, and event discovery and participation by volunteers. The system also supported participant management, attendance verification within a defined time screen, and impact-point-based participation tracking.

In addition, the scope included user profile management, public profile viewing, community interaction through likes and comments, and social connectivity through following or friend relationships. These features ensured both operational functionality and user engagement within a single platform.

From a technical perspective, the project scope included front-end development using Flutter, Firebase Authentication for user management, Cloud Firestore for data storage, and Firebase Storage for handling event-related media. The system also incorporated security rules to ensure protected and reliable data operations.

However, the project did not include payment integration, real-time chat systems, advanced analytics, and large-scale deployment optimizations. These aspects were identified as future enhancements.

Overall, the scope was defined to deliver a complete, reliable, and user-friendly core platform for event management, attendance tracking, and community interaction within the given project timeline.

Chapter 02

2.1 Requirement gathering techniques

The requirements for the GoodDeeds system were gathered through a structured and iterative process that combined multiple techniques to ensure completeness and alignment with user needs. Since the system supports two primary user groups, volunteers and organizers, the requirement gathering process focused on identifying differences in user goals, tasks, and access levels from the early stages of development.

Initial requirements were identified through discussions with potential users, which highlighted common issues in existing community service coordination methods, such as fragmented communication, weak attendance verification, and limited visibility of participation records. These discussions helped establish the need for a unified system that supports role-based access, event lifecycle management, and activity tracking.

Further analysis was conducted by modelling the two core user roles, volunteer, and organizer. For each role, expected actions were identified and prioritized. Volunteers were mainly responsible for browsing and joining events, while organizers were responsible for creating, managing, and monitoring events. This helped clearly define system functionalities and reduce ambiguity in feature design.

In addition, existing systems such as manual record-keeping, social media platforms, and messaging applications were reviewed to identify operational gaps. These approaches lacked structured participant management, reliable attendance tracking, and integrated data handling. The findings from this review directly contributed to defining the system problem and shaping the core functional requirements.

Use-case scenarios were then developed to represent key system processes, including user registration, authentication, role-based navigation, event participation, attendance verification, and social interactions. This approach helped translate user expectations into clear and implementable functional requirements.

Early interface designs and screen flow sketches were also created to validate navigation logic and user experience. Feedback obtained during this stage helped refine layout structure, improve usability, and ensure smooth user journeys across both volunteer and organizer workflows.

The requirement gathering process was further supported by iterative validation during development. Features were continuously tested against initial requirements, and necessary refinements were made in each development cycle. This ensured that the implemented system remained aligned with the original objectives.

Finally, in addition to functional requirements, non-functional aspects such as security, performance, usability, reliability, and maintainability were also identified and treated as essential requirements. These considerations guided system design decisions and ensured the development of a stable and efficient application.

2.2 Functional and non- Functional requirements

❖ Functional Requirements

Functional requirements describe the core operations and features of the system. The GoodDeeds application includes the following functional requirements:

- The system shall allow users to register, create a user profile, and sign in securely using email and password authentication.
- The system shall provide password reset and sign-out functionality to complete the authentication lifecycle.
- The system shall provide role-based access control and route users to appropriate dashboards based on their role (Organizer or Volunteer).
- The system shall allow volunteers to browse available events, view event details, join events, and leave events.
- The system shall allow volunteers to track their participation status and view their event history.
- The system shall allow organizers to create, edit, and manage events.

- The system shall enable organizers to upload and manage event images.
- The system shall allow organizers to view and manage participant lists.
- The system shall support attendance check-in within a controlled verification period and allow organizers to verify attendance.
- The system shall allocate, and track impact points based on user participation.
- The system shall provide profile management, including public profile visibility and activity history.
- The system shall allow users to view completed and pending events.
- The system shall support social interaction features such as likes, comments, and following/friends' functionality.

❖ **Non - Functional Requirements**

The non-functional requirements state the qualities of the system and its limitations. The GoodDeeds application includes the following non-functional requirements:

- The system shall maintain acceptable performance under normal usage conditions, including smooth UI behavior, practical load times, and near real-time updates.
- The system shall ensure secure data storage and retrieval using Firebase services and Firestore security rules.
- The system shall enforce security through authenticated access, role-based authorization, and backend rule validation for protected operations.
- The system shall provide a clear, consistent, and user-friendly interface with intuitive navigation and meaningful feedback messages.
- The system shall ensure reliability in critical operations such as authentication, event updates, attendance tracking, and point management.
- The system shall include proper error handling mechanisms to manage system or service failures.
- The system shall maintain accurate and consistent data, especially for participation records and impact points.
- The system shall be maintainable, with modular code organization and clear separation of models, services, and UI components to support future enhancements.

- The system shall support cross-platform compatibility across Flutter-supported environments and varying mobile screen sizes.
- The system shall ensure high availability and accessibility of the application.

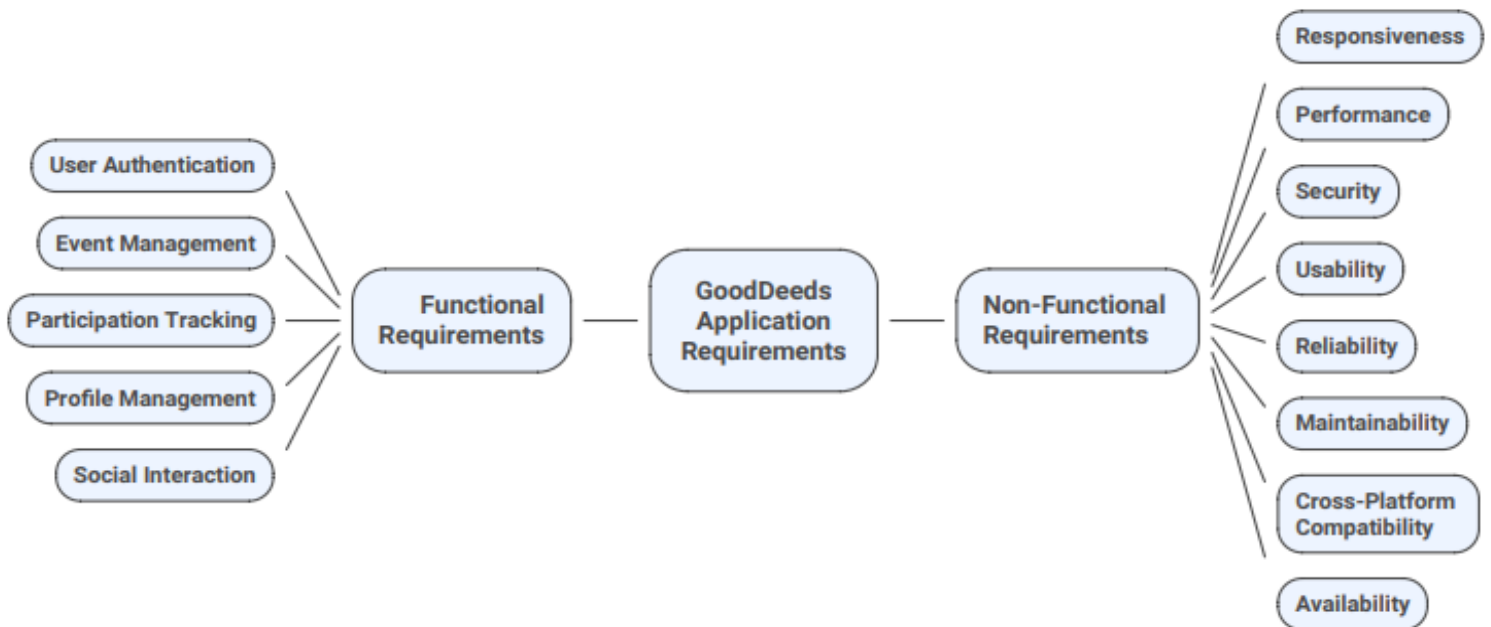


Figure 2.2.1: GoodDeeds application requirements.

2.3 Features of the Application

The GoodDeeds application provides a range of features designed to support efficient community service coordination, event management, and user engagement. The key features of the system are as follows:

- **User Authentication and Account Management**

The system includes registration, login, logout, and password reset functionality with secure email and password authentication. It also supports role-based routing after login.

- **Role-Based Dashboard**

The application provides separate dashboards for organizers and volunteers, allowing access to role-specific features and functionalities.

- **Event Creation and Management**

Organizers can create, edit, and manage events using dedicated tools, including uploading event banners, and updating event details.

- **Event Discovery and Community Browsing**

Volunteers can browse available events, search for activities, view event details, and explore community content through structured discovery screens.

- **Event Participation and Attendance Management**

Users can join and leave events, while organizers can monitor participants and manage attendance through a controlled check-in process. The system also tracks participation status.

- **Time-Based Event Control and Attendance Policy**

A key feature of the system is time-based event control. Events are automatically treated as expired after their scheduled date and time, allowing volunteers to clearly distinguish between active and past activities. Attendance verification follows a strict rule: check-in is permitted only from the event start time up to 48 hours after the event, and updates outside this period are restricted. This ensures fair participation tracking, prevents late manual modifications, and improves the reliability of impact point records.

- **User Profiles and Public Profiles**

The system provides profile management features, including editable personal profiles and public profiles that display user activity history and contributions.

- **Impact Points Tracking**

The application tracks user participation and awards impact points to reflect contributions and engagement.

- **Social Interaction Features**

The system supports likes, comments, and a following/unfollowing feature, enabling users to interact, build connections, and engage within the community.

- **Data Persistence and Real-Time Updates**

The system uses Firebase services to ensure reliable data storage and near real-time updates for events, user activities, and engagement data.

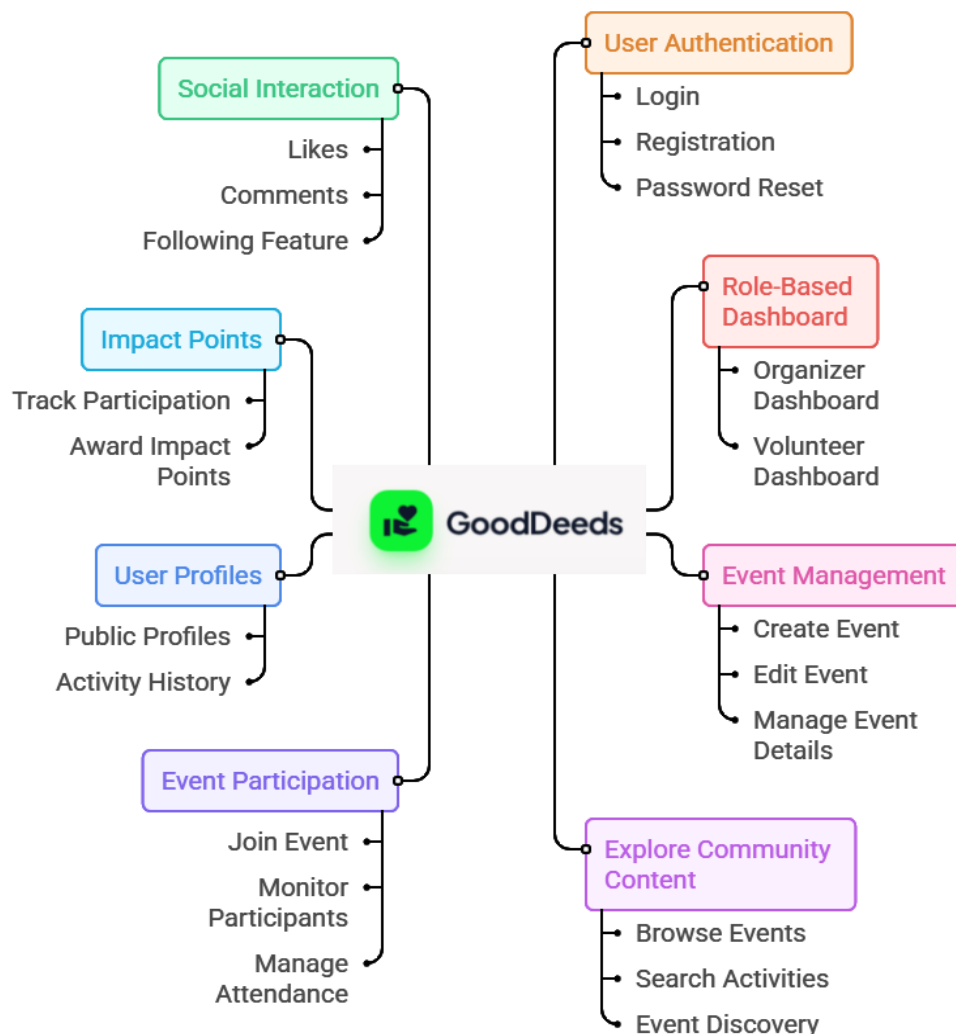


Figure 2.3.2: Feature of the GoodDeeds App.

Chapter 03

3.1 Use case diagram

The Use Case Diagram for the GoodDeeds system illustrates the interactions between users and the system, highlighting the main functionalities available to different user roles. The system primarily consists of two actors: **Volunteer** and **Organizer**, each with distinct responsibilities and access levels within the application.

The Volunteer actor interacts with the system to perform actions such as registering an account, logging into the system, browsing available events, viewing detailed event information, joining events, and tracking their participation history. Volunteers are also able to manage their profiles, view public profiles of other users, and engage with the community through social features such as likes, comments, and following.

The Organizer actor has additional privileges that allow them to manage events within the system. These actions include creating new events, editing event details, uploading event banners, viewing participant lists, and verifying attendance through a controlled check-in process. Organizers are responsible for ensuring that event records are accurate, and that participation is properly tracked.

Common features shared by both actors include community feed interaction, profile management, and authentication. Role-based access control is ensured by the system, allowing each user to access just the features that are relevant to their role.

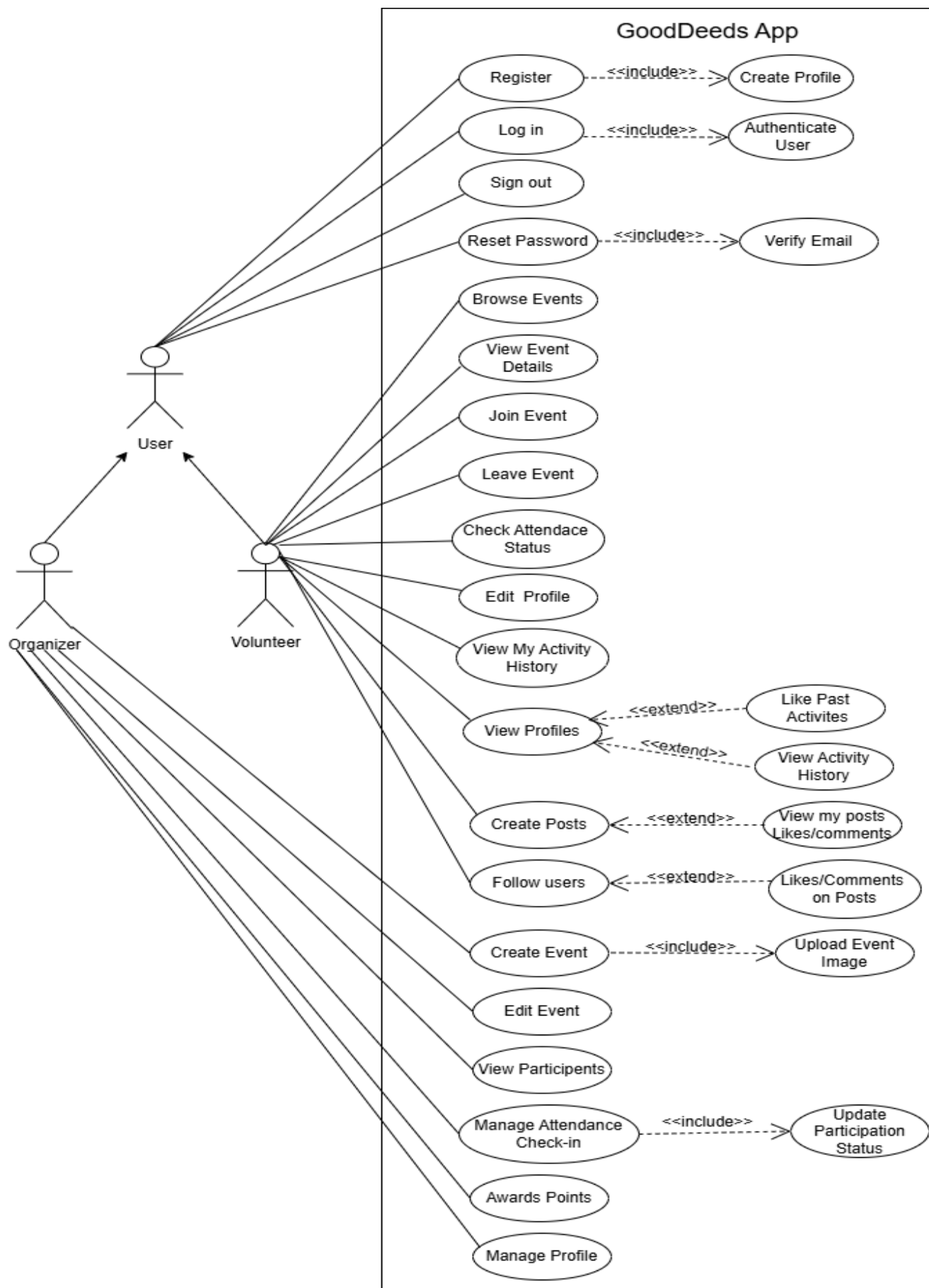


Figure 3.1.1: Use Case Diagram.

3.2 High – Level diagram

The high-level diagram of the GoodDeeds system shows a Flutter-based client application communicating with Firebase services to deliver the main functionality of the platform. The system follows a client-server architecture, where the mobile application acts as the front-end interface and Firebase provides the backend infrastructure.

The user interface layer handles key functionalities including registration, login, event browsing, event creation, attendance management, profile viewing, and social interaction features such as likes, comments, and following. Users interact with the system through these screens based on their roles.

Firebase Authentication manages secure user registration and login processes. Cloud Firestore is used to store and retrieve application data, including user accounts, user profiles, event details, participation records, attendance data, comments, likes, and following relationships. Firebase Storage is used to store uploaded event images and media files associated with events.

The diagram also illustrates the implementation of role-based access control, where system logic ensures that users are directed to appropriate interfaces and functionalities based on their roles as volunteers or organizers. This separation ensures proper access to features such as event participation for volunteers and event management for organizers.

Communication between the Flutter front-end and Firebase backend occurs through secure API interactions, enabling real-time data synchronization and efficient system performance.

Overall, the high-level diagram demonstrates how the front-end application, backend services, and data storage components work together to support the complete system workflow, including event management, participation tracking, attendance verification, and social engagement.

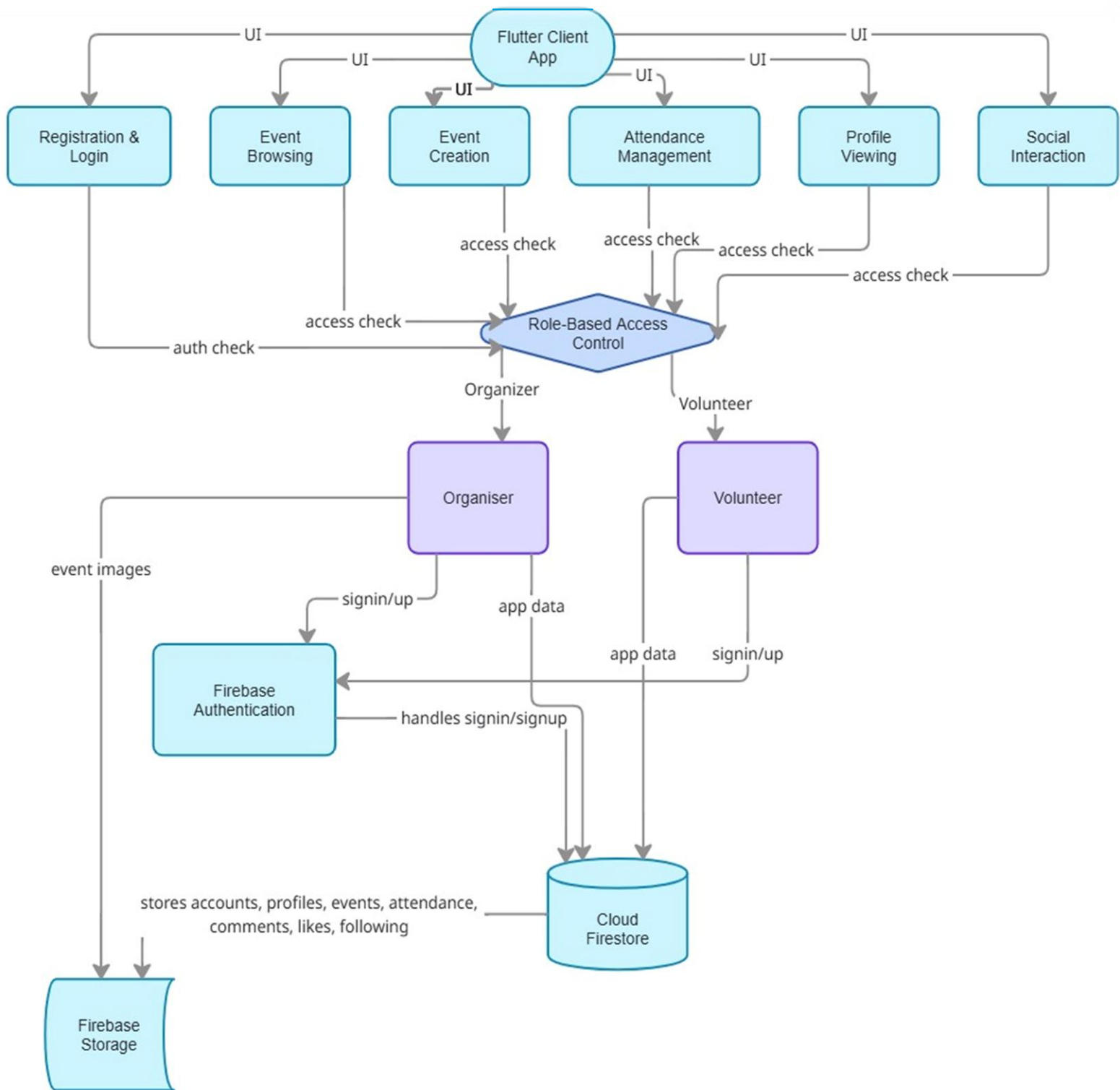


Figure 3.3.3: High level Diagram.

3.3 EER Diagram

The Enhanced Entity Relationship (EER) Diagram of the GoodDeeds system represents both event management and social engagement, incorporating role-based behavior and associative entities.

The central entity, **User**, stores core details such as user ID, name, email, role, and impact points. Each user can act as a volunteer or organizer. It maintains one-to-one relationships with **UserProfile** (bio, image, location) and **UserLookup** (normalized fields for efficient searching).

In event management, an organizer (User) has a one-to-many relationship with **Event**, where each event includes attributes such as title, description, location, category, date, and status. The many-to-many relationship between User and Event is resolved through **EventParticipation**, which records participation status, attendance, check-in, and reward points.

For social interaction, **EventLike** captures event likes, while **Post** and **Comment** support user-generated content and interactions. The **Follow** entity models a recursive many-to-many relationship among users.

Overall, the model ensures proper normalization, supports role-based functionality, enables accurate tracking, and provides scalable social features.

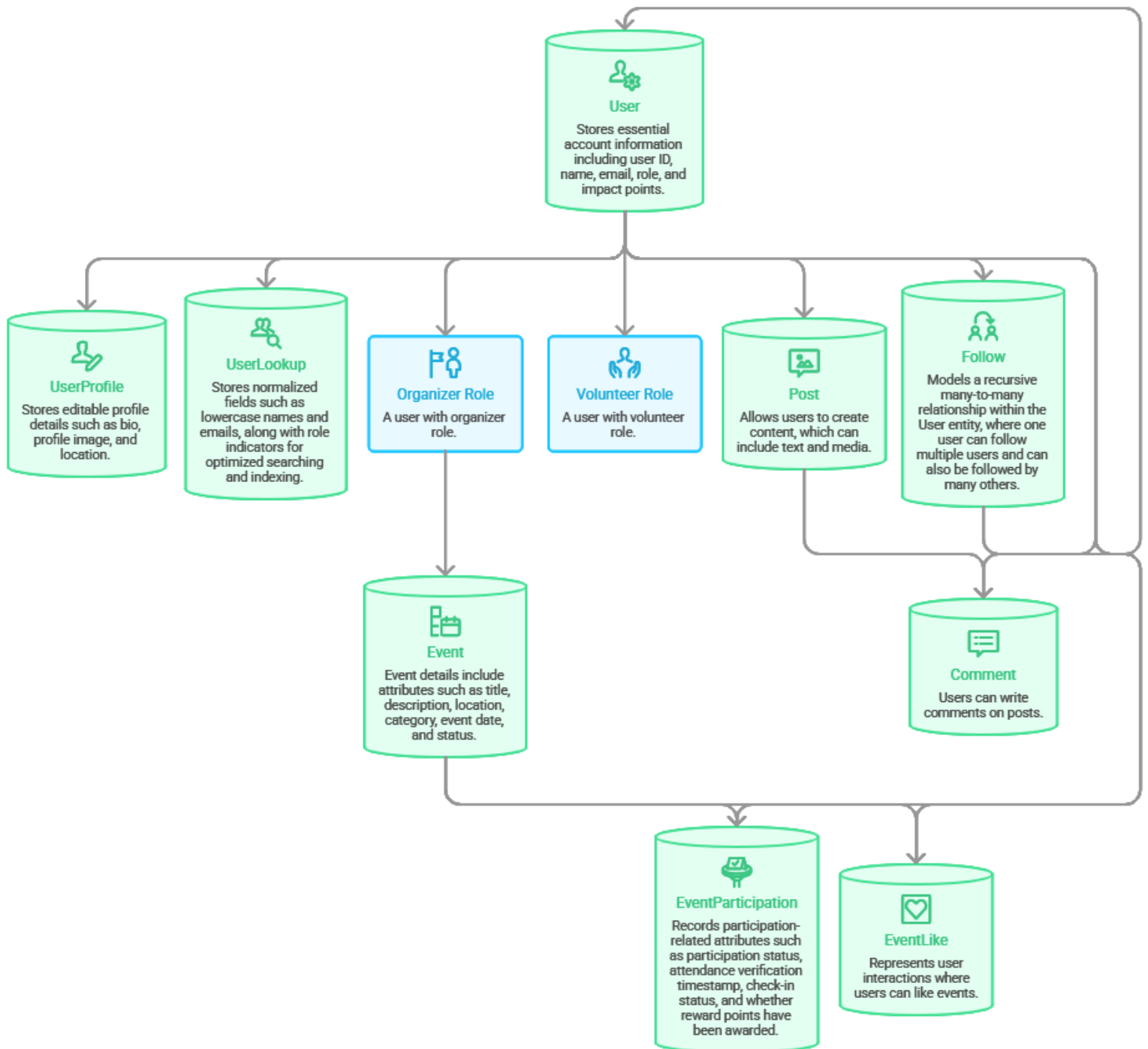


Figure 3.3.3: EER: Illustrates the EER Diagram of the GoodDeeds app.

3.4 User interface of the developed System

The following figures illustrate the key user interface screens of the developed GoodDeeds system.

- Authentication Screens

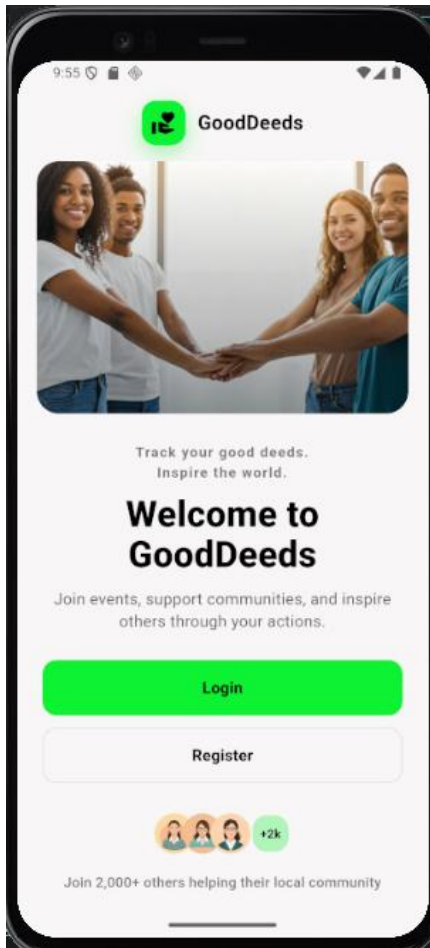


Figure 3.4.1: Welcome Screen

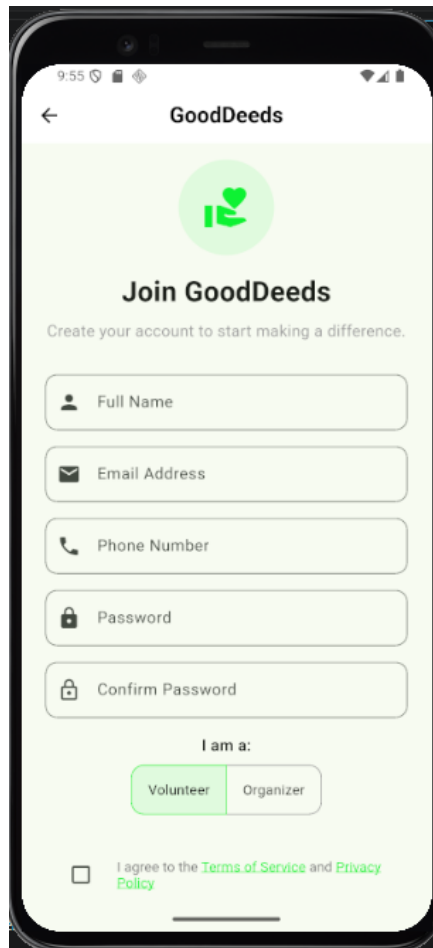


Figure 3.4.2: Registration Screen

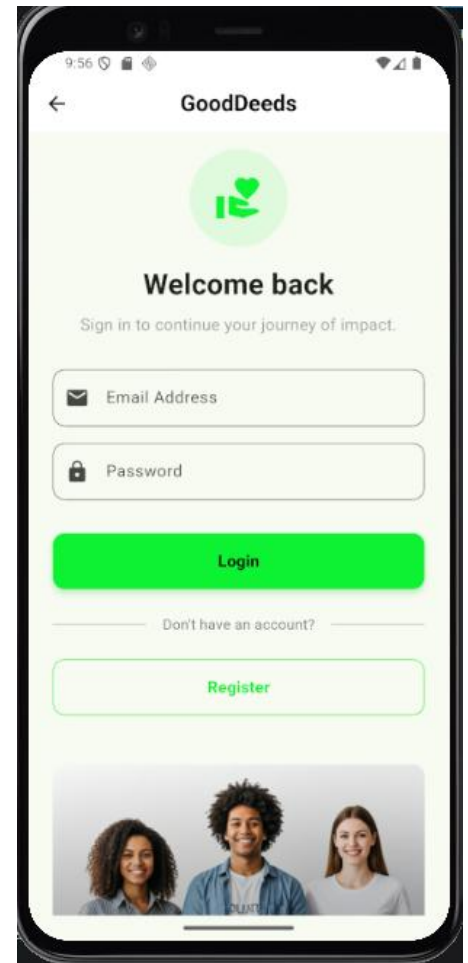


Figure 3.4.3: Login Screen

thesanya@gmail.com	✉	31 Mar 2026	2 Apr 2026	NrJGyOyoLGM7SRZDrkf4eI...
dinuka@gmail.com	✉	31 Mar 2026	2 Apr 2026	yfIRVO7qEDVZF8Ln5UWqY9...
kasundiranaweera@g...	✉	31 Mar 2026	3 Apr 2026	bjjDhUSli1h2bA5B01yMI7A4P8...
raskumara@gmail.com	✉	31 Mar 2026	3 Apr 2026	BqL3ByYpH3a1amrK8AllvQlJe...
supunyanaweera@g...	✉	29 Mar 2026	2 Apr 2026	ZjFsFwETTibVuejQyok6qDFQ...

Figure 3.4.4: Firebase Authentication.

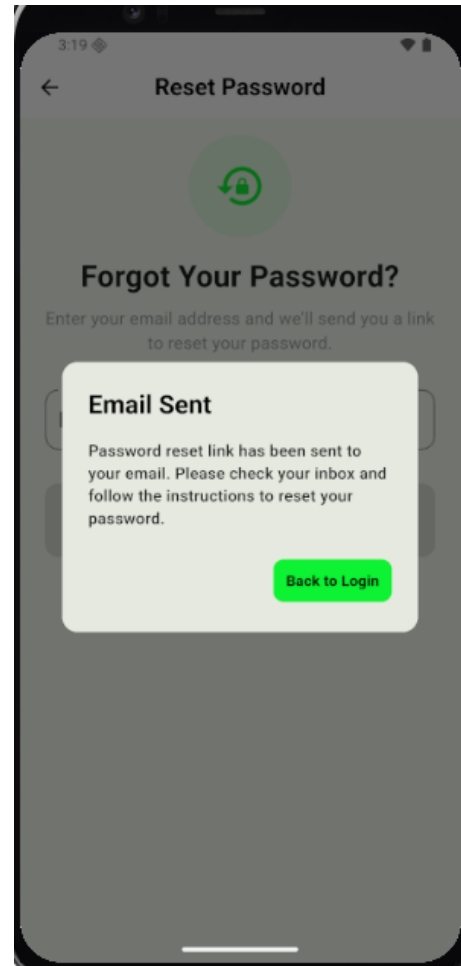
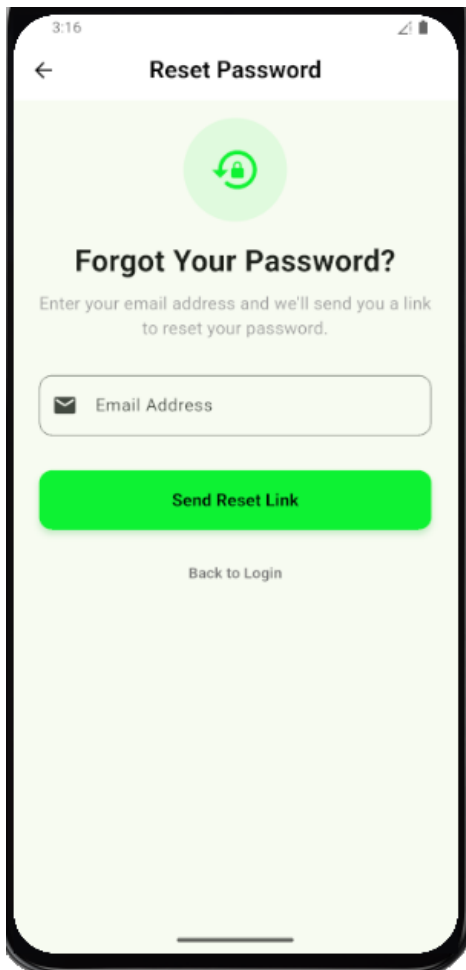


Figure 3.4.5: Forgot Password Screen.

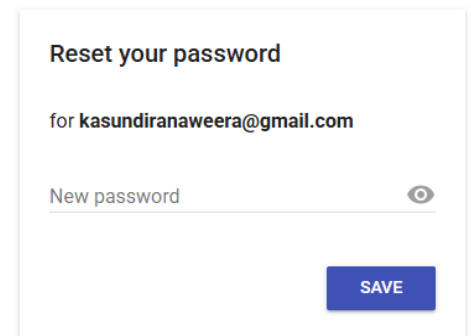
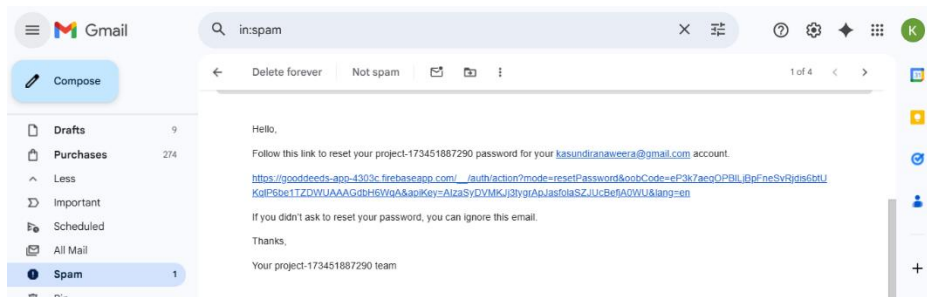


Figure 3.4.6: Password Reset Email Received.

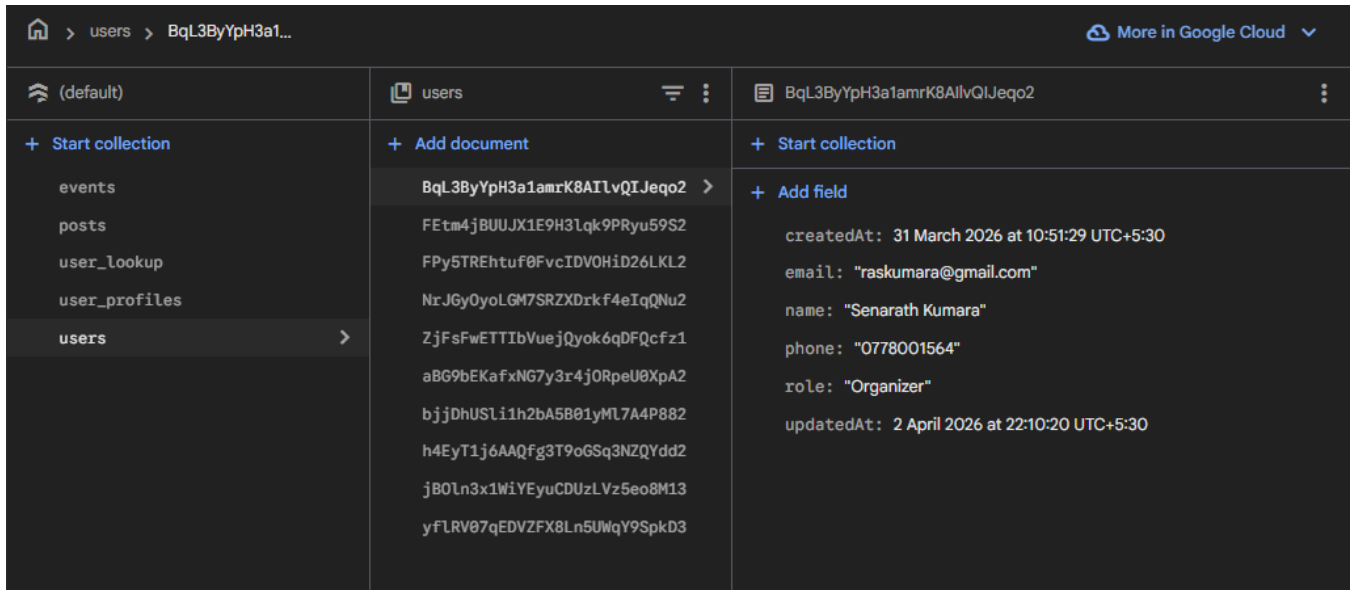


Figure 3.4.7: Firebase Firestore Database Structure.

- Organizer Screens

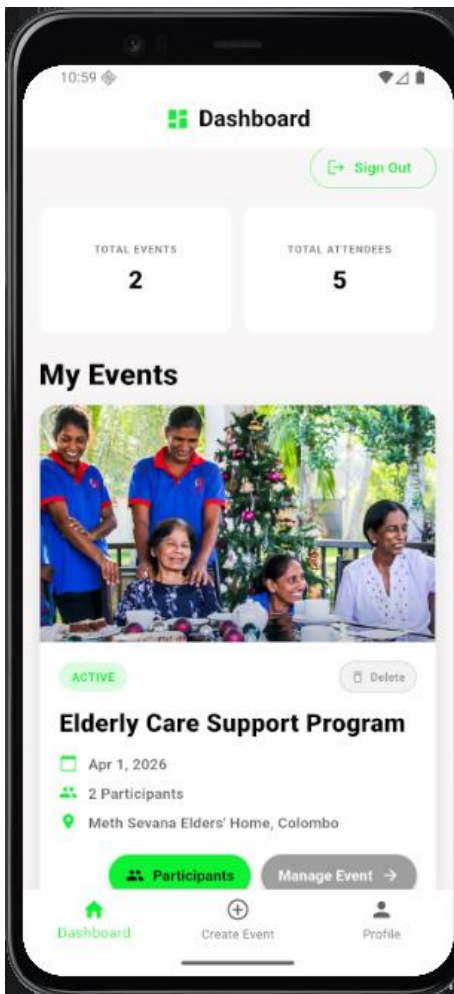


Figure 3.4.8: Organizer dashboard Screen

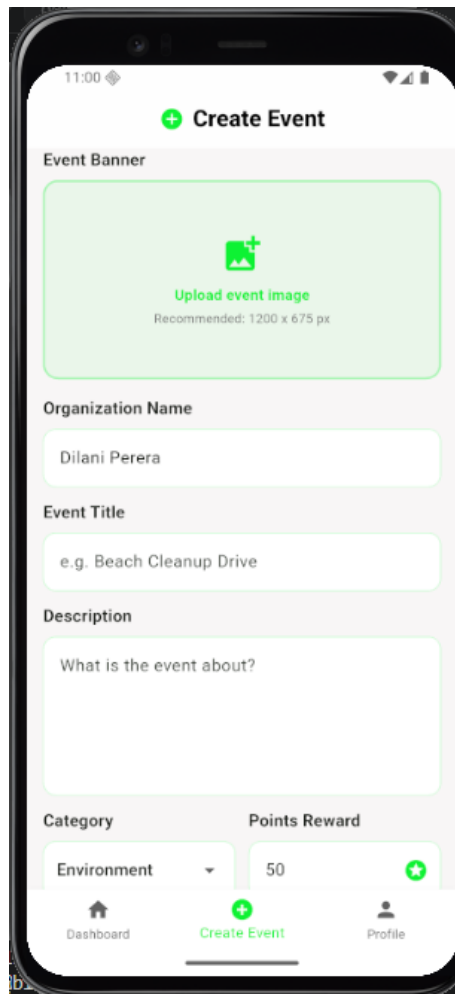


Figure 3.4.9: Create Event Screen

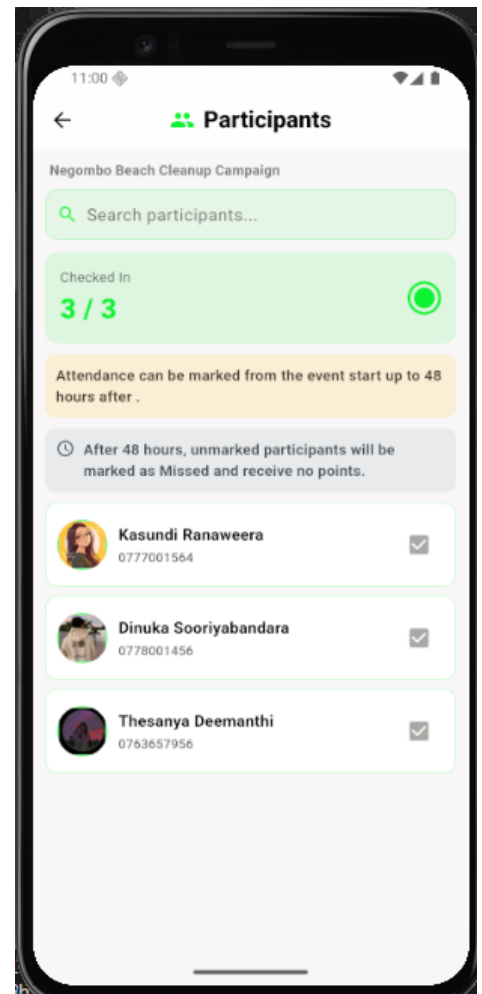


Figure 3.4.10: Participants Screen

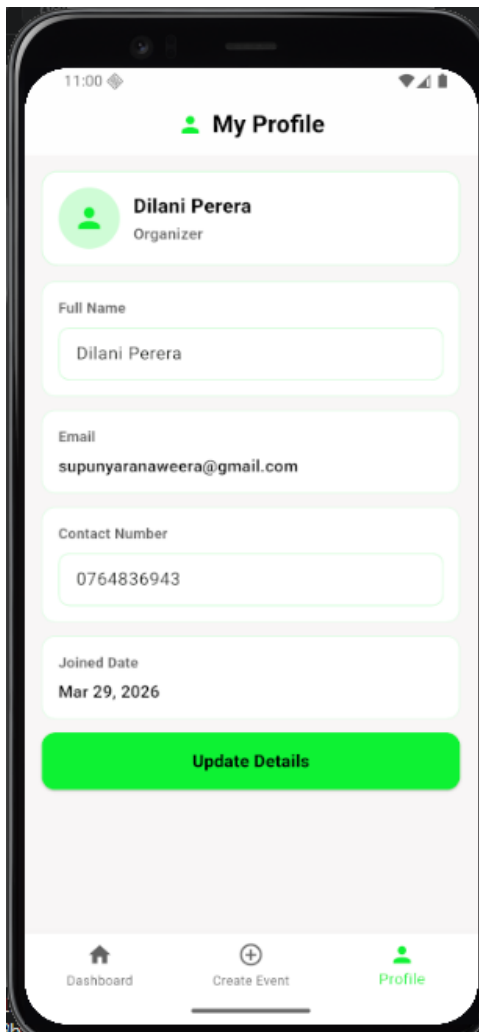


Figure 3.4.11: Organizer Profile Screen

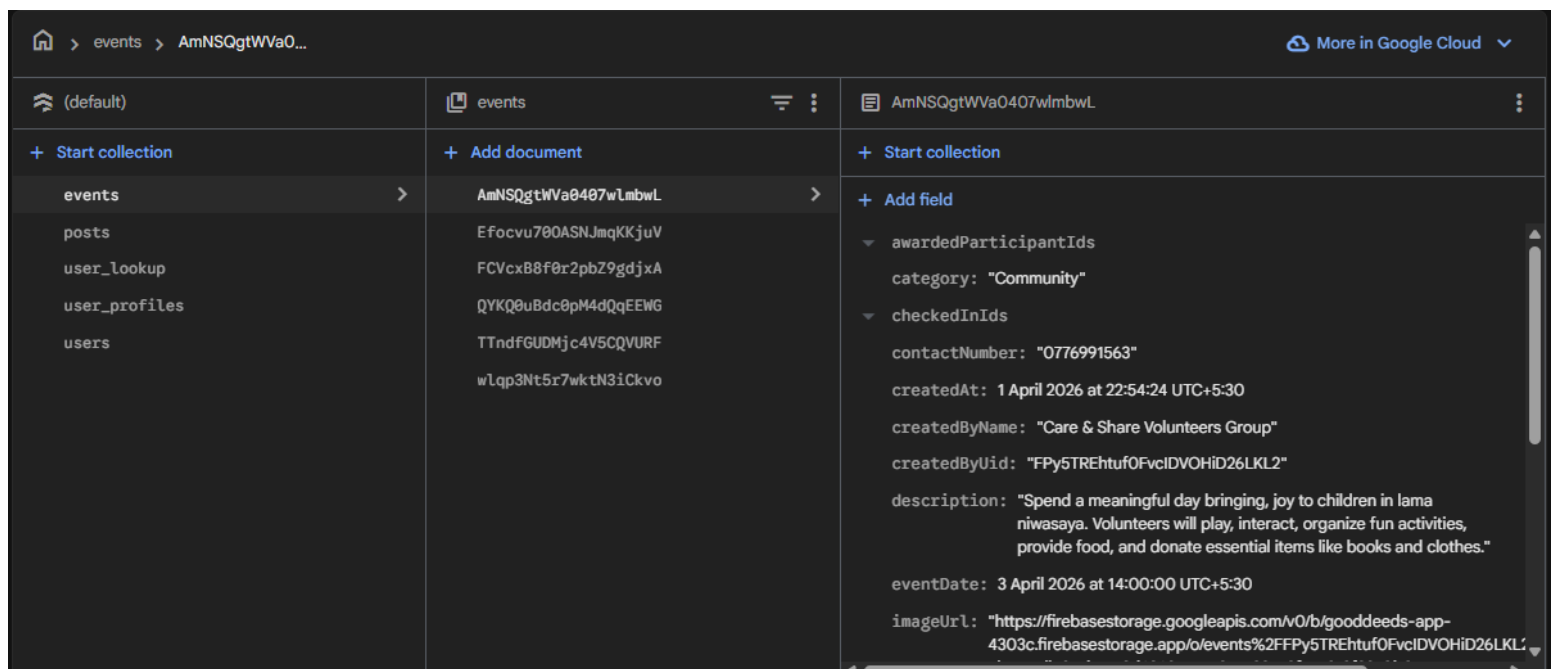


Figure 3.4.12: Firestore Events Collection

- Volunteer Dashboard

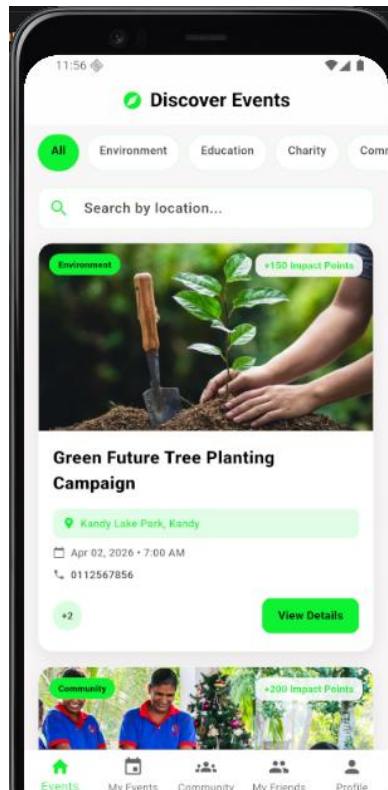


Figure 3.4.13: Discover Events



Figure 3.4.14: Event Details

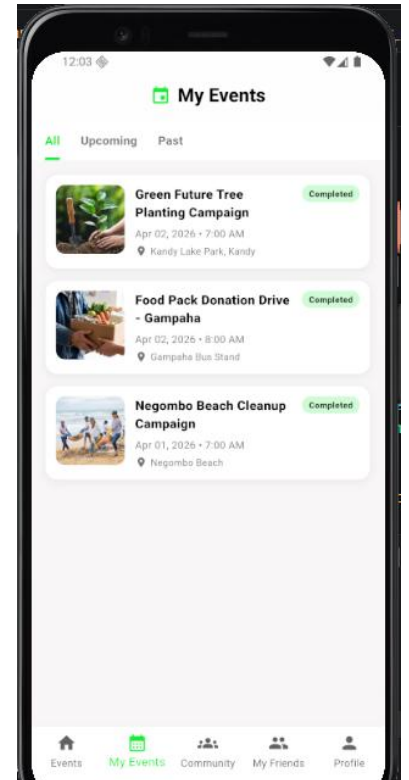


Figure 3.4.15: My Events

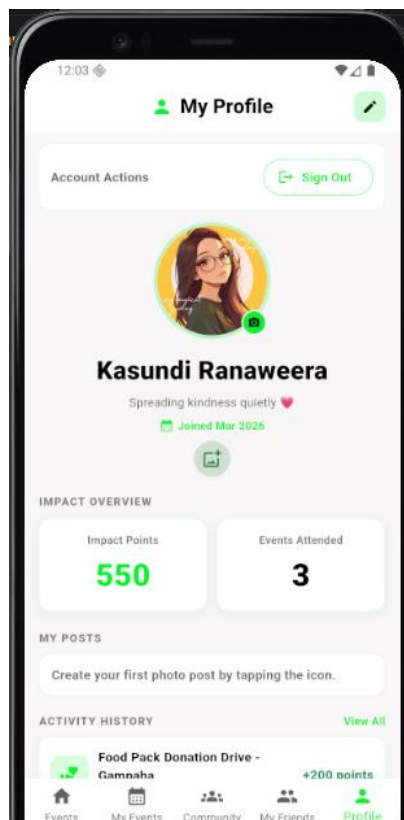


Figure 3.4.16: User Profile

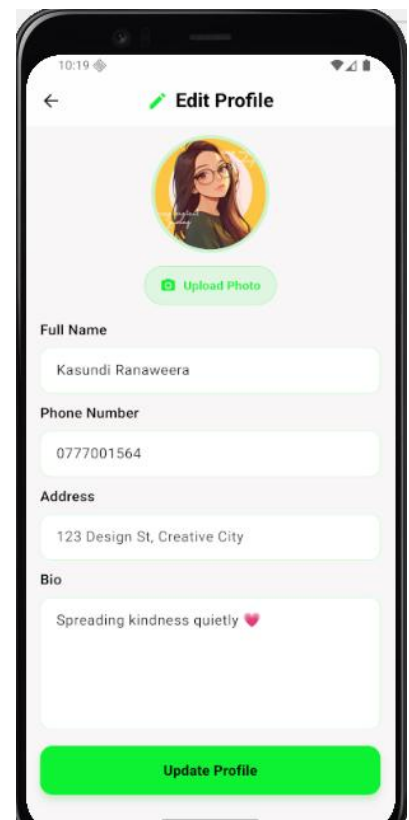


Figure 3.4.17: Edit Profile

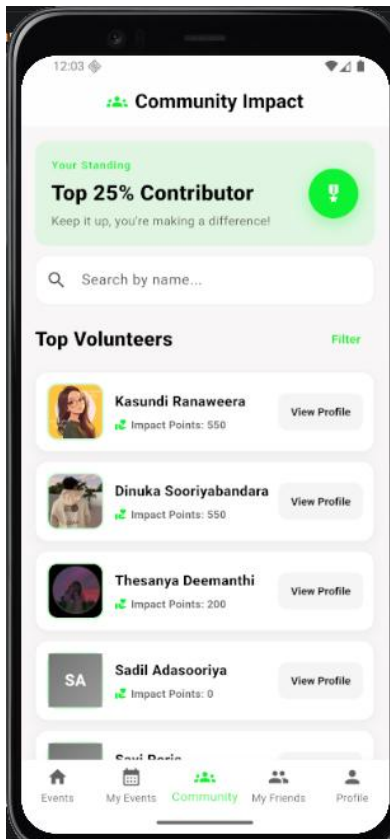


Figure 3.4.18: Community Screen

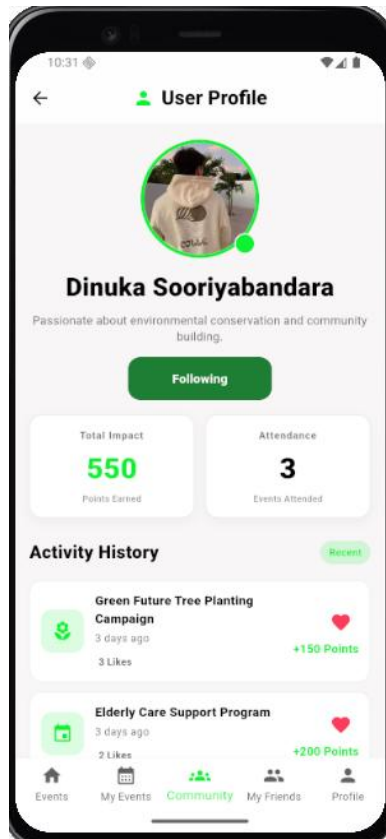


Figure 3.4.19: Public Profile

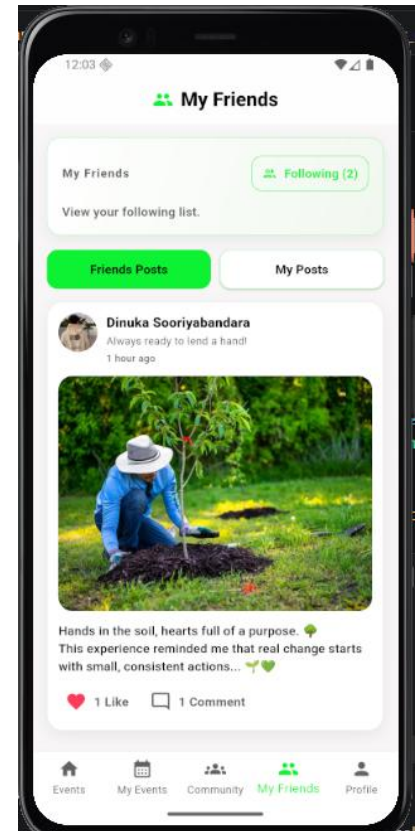


Figure 3.4.20: My Friends Screen



Figure3.4.21: My Friends - my post

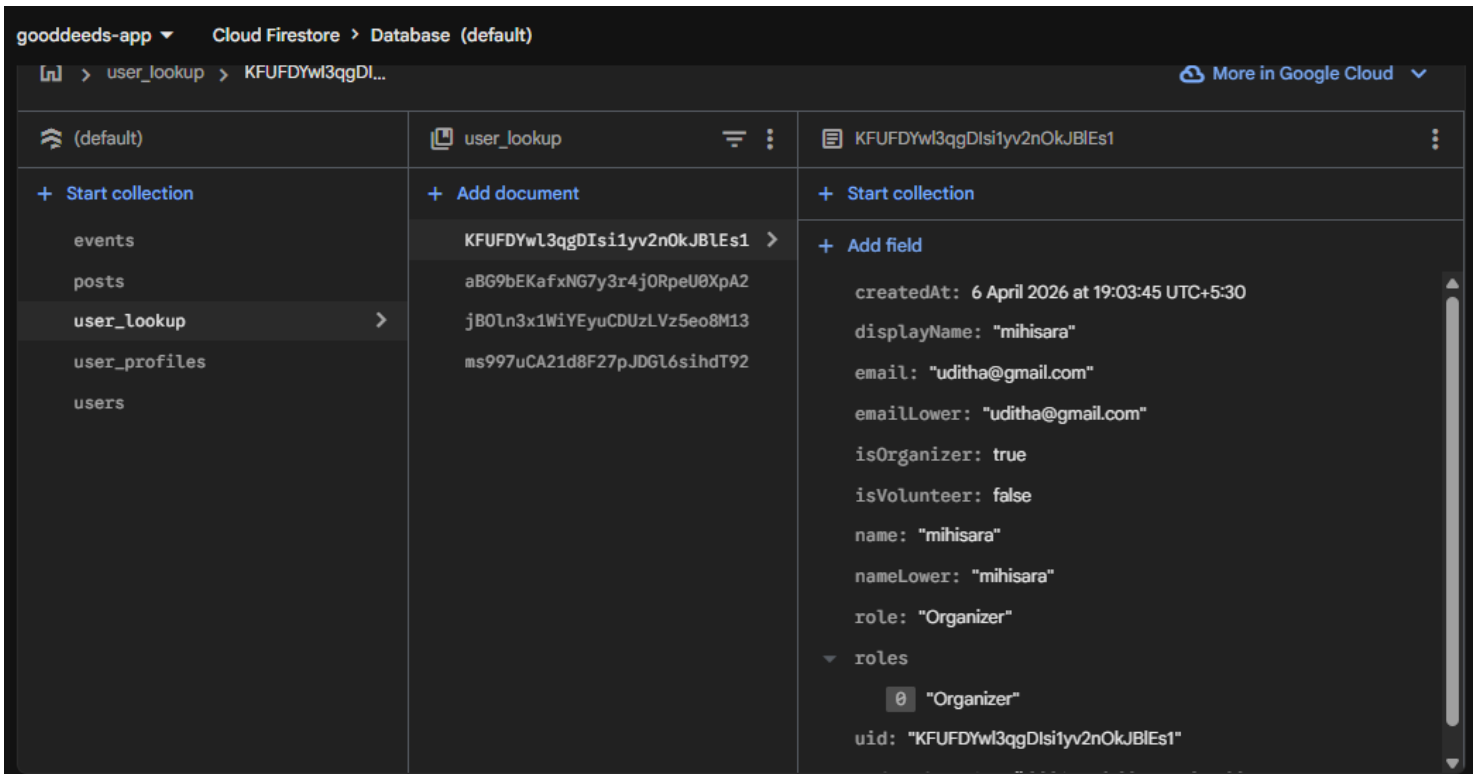


Figure 3.4.22: Firebase firestore user_lookup collection.

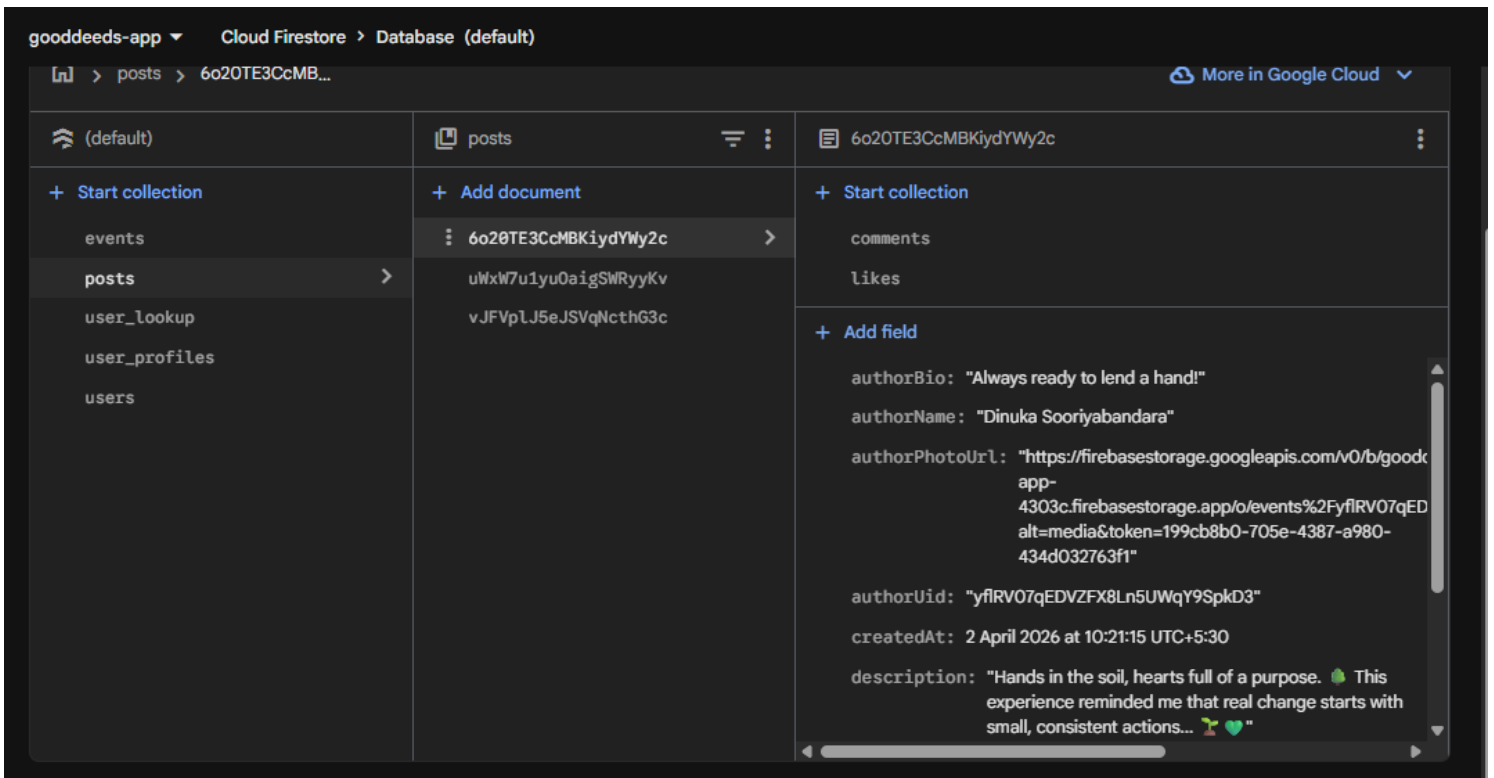


Figure 3.4.23: Firebase firestore post collection.

Chapter 04

4.1 Development Methodology

The GoodDeeds project was developed using an **Agile methodology**, following an iterative and incremental approach that combined feature-driven development with continuous testing and refinement. The development process was organized into short cycles (sprints), where individual features or feature groups were planned, implemented, tested, and reviewed before progressing to the next cycle. This approach enabled the team to deliver working functionality progressively, identify design issues early, and refine requirements based on practical implementation insights.

The methodology prioritized core operational features such as authentication, event creation, event participation, and attendance tracking followed by engagement and social features, including profiles, comments, likes, and following. Each development cycle included design review, code implementation, unit testing, integration with Firebase services, and validation against requirements. This sequencing ensured that foundational systems were stable before building higher-level functionality.

Version control using GitHub was central to the development process, supporting concurrent development, code review, and rollback when necessary. Regular commits documented the evolution of features and enabled team members to track progress and resolve conflicts effectively. The iterative Agile approach also supported collaborative debugging, allowing the team to quickly identify issues, apply fixes, and re-test without delaying overall development.

By combining short Agile development cycles with continuous testing and Firebase backed validation, the methodology reduced technical risks, improved code quality, and ensured that the delivered system remained aligned with project goals throughout development.

4.2 Technologies and tools used

Technology / Tool	Purpose	Description
Flutter and Dart	Front-end development	Used to build the mobile application from a single codebase with a responsive UI using Flutter's widget system.
Firebase Authentication	User authentication	Provides secure email and password-based registration, login, and account management.
Cloud Firestore	Database	NoSQL database used to store user data, events, attendance records, comments, likes, and following relationships in real time.
Firebase Storage	Media storage	Used to store and retrieve uploaded event images and other media files securely.
Firebase Security Rules	Data security	Applies role-based access control mechanisms to ensure that sensitive information is safe from any form of intrusion.
Visual Studio Code	Development tool	Used for coding, debugging, and managing Flutter application code.
Android Studio	Development & testing	Used for Android emulator testing, debugging, and application deployment.
Git & GitHub	Version control	Used for source code management, branch tracking, collaboration, and maintaining commit history.
Flutter Build Tools & Config Files	Deployment support	Used for building, configuring, and deploying the application across different platforms (mobile and web).

4.3 Future Implementation

Although the GoodDeeds application successfully implements the core features required for event management and community engagement, future work can focus on expanding user engagement, improving organizer support, and strengthening automation.

One key enhancement is the integration of push notifications for event updates, reminders, and attendance deadlines. This would help users respond quickly to new activities and stay informed about schedule changes, thereby increasing participation and engagement.

The event search and discovery process can also be improved by introducing advanced filtering options, category-based ranking, and location-based suggestions. These features would allow users to find relevant events more efficiently and enhance their overall experience.

For organizers, a more advanced dashboard can be developed to provide insights such as participation trends, attendance summaries, and event performance statistics. This would support better planning and decision-making for future events.

Attendance management can be further improved by introducing QR code or barcode-based check-in systems. This would make the verification process faster, more accurate, and more reliable compared to manual methods.

Social features of the platform can also be extended by adding richer profile interactions, direct messaging between users, and improved moderation tools for comments and posts. These enhancements would strengthen community engagement and ensure a safer platform environment.

In addition, analytics features can be implemented to track volunteer activity, reward points, and overall event outcomes. These insights would help both users and organizers better understand their contributions and impact.

Overall, these future improvements will make the GoodDeeds system more scalable, interactive, and effective, providing greater value to both volunteers and organizers.

Chapter 05

5.1 Individual contribution

1. Ranaweera Ranaweera – 10965595

Established the project foundation by initializing the Flutter application and structuring the overall architecture while leading and coordinating the development process. Defined core models and constants, implemented the Firebase service layer, and handled authentication integration. Developed key screens including Welcome, Register, Login, Home, Forgot Password, and created the My Events feature, enabling users to view and manage their events. Additionally, led and guided other team members in developing their screens, ensuring consistency with the overall system design and functionality. Took full responsibility for the development process, including making key development decisions, managing the workflow, and maintaining overall control of the project implementation.

Adasooriya Adasooriya - 10965435

Contributed to the development of the social interaction module by implementing the Community screen, Public Profile screen, and My Friends screen. This work enabled users to discover other users, view public profiles, and build connections through following and interaction features, improving overall user engagement within the platform. Additionally, basic navigation and user-friendly layouts were designed to ensure a smooth experience, while integrating dynamic data to keep user information and interactions up to date.

Senarath Bandara - 10965463

Developed the Organizer Dashboard and Participants screens, allowing organizers to monitor events and manage participants effectively. This included displaying participant lists, handling attendance tracking, and supporting organizer-side decision-making through structured data presentation.

2. Ramanayake Mihisara - 10965569

Implemented the Create Event and Manage Event screens, enabling organizers to handle the full event lifecycle. This included event creation, updating event details, and managing event-related data, ensuring flexibility and control over event operations. Designed the interfaces to be user-friendly and responsive, allowing organizers to easily modify or maintain events.

3. Bengamu Perera - 10965579

Developed the Organizer Profile, Public Profile screen, and Comments features. This contribution supported organizer profile presentation, enhanced profile visibility, and improved user engagement through interactive discussion functionality.

4. Rammandalage Rammandala - 10965589

Implemented the Discover Events and Event Details screens, allowing users to browse available events, view detailed information, and interact with event content. This supported the volunteer side experience by making event discovery and participation more accessible and user-friendly.

5. Kapuge Dewmini – 10965483

Developed the User Profile and Edit Profile features, enabling users to view and update their personal information. This supported user identity management and ensured that profile data remained accurate and customizable within the system.

5.2 GitHub Commit History

GitHub repository link: <https://github.com/KasundiRanaweera/GoodDeeds-Main-App>

GoodDeeds-Main-App Public

11 Branches 0 Tags

Go to file Add file Code

KasundiRanaweera Flutter CI ✓ d264ee8 · 17 hours ago 35 Commits

.github/workflows	Flutter CI	17 hours ago
android	first commit	last week
docs/screenshots	first commit	last week
functions	first commit	last week
ios	first commit	last week
lib	Merge pull request #11 from KasundiRanaweera/feature/co...	2 days ago
linux	first commit	last week
macos	first commit	last week

About

No description, website, or topics provided.

Readme Activity 0 stars 0 watching 0 forks

Releases

No releases published [Create a new release](#)

Packages

[GitHub repository Overview]

Branch	Updated	Check status	Behind	Ahead	Pull request	
feature/comment	2 days ago		25	0	#11	...
comment-screen	2 days ago		26	0		...
feature/discover-events-and-details	2 days ago		6	0	#10	...
feature/community-public-screens	3 days ago		18	0	#9	...
my-events	3 days ago		15	0	#8	...
feature/user-profile-edit	4 days ago		19	0	#5	...
orgsnizer-profile	5 days ago		26	0	#2	...
feature/organizer-dashboard-participant-screens	5 days ago		26	0	#3	...
organizer-event	5 days ago		26	0	#4	...
home-screen	5 days ago		28	0	#1	...

[Branch Structure]

The screenshot displays the GitHub Actions interface for the repository 'GoodDeeds-Main-App'. The left sidebar shows the 'Actions' tab with a 'New workflow' button and a list of workflow categories: Flutter CI, Management, Caches, Attestations, Runners, Usage metrics, and Performance metrics. The main area is titled 'All workflows' and shows 'Showing runs from all workflows'. A search bar 'Filter workflow runs' is present. Below, a table lists workflow runs. One run is visible: 'Flutter CI' (status: success), triggered by 'Flutter CI #1: Commit d264ee8 pushed by KasundiRanaweera' on the 'main' branch. It ran on 'Apr 7, 10:52 PM GMT+5:30' and took '1m 34s'.

[GitHub action workflow]

5.3 Project Source Code Link

https://liveplymouthac-my.sharepoint.com/:f:/g/personal/10965435_students_plymouth_ac_uk/IgBKX_gdNwG2QYr9ax2UKXT9AZfef-qNEPfc2fWwvWa1ObU?e=8JAJL0

Reference List

Sandhu, R.S. et al. (1996) 'Role-Based Access Control Models', *IEEE Computer*, 29(2), pp. 38–47. Available at:

<https://csrc.nist.gov/projects/role-based-access-control>

Ferraiolo, D.F. and Kuhn, D.R. (1992) 'Role-Based Access Controls', *15th NIST-NCSC National Computer Security Conference*. Available at:

<https://nvlpubs.nist.gov/nistpubs/Legacy/IR/nistir6192.pdf?>

Kaplan, A.M. and Haenlein, M. (2010) 'Users of the world, unite! The challenges and opportunities of social media', *Business Horizons*, 53(1), pp. 59–68. Available at:

<https://www.sciencedirect.com/science/article/abs/pii/S0007681309001232>

Ellison, N.B., Steinfield, C. and Lampe, C. (2007) 'The benefits of Facebook “friends”', *Journal of Computer-Mediated Communication*, 12(4), pp. 1143–1168. Available at:

<https://academic.oup.com/jcmc/article/12/4/1143/4582961>

Hager, M.A. and Brudney, J.L. (2011) 'Problems recruiting volunteers', *Nonprofit Management & Leadership*, 22(2), pp. 137–157. Available at:

<https://onlinelibrary.wiley.com/doi/10.1002/nml.20054>

Hyde, M.K., Dunn, J., Bax, C. and Chambers, S.K. (2016) 'Episodic volunteering and retention: An integrated theoretical approach', *Nonprofit and Voluntary Sector Quarterly*. Available at:

<https://journals.sagepub.com/doi/10.1177/0899764014558934>